

VIT BHOPAL UNIVERSITY

BUILD YOUR OWN PROJECT

SMART STUDENT ACADEMIC MANAGEMENT SYSTEM

Web-Based Academic Management Application

Submitted By	Shriya Parmar
Registration Number	25BAI10307
Program	B.Tech – Artificial Intelligence & Machine Learning
Academic Year	2026–2027

Project Report

LIVE DEMO

[Open Live Website](#)

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Scope of the Project
5. Functional Requirements
6. Non-Functional Requirements
7. System Modules
8. Technology Stack
9. System Architecture
10. UML and Design Views
11. Implementation
12. Application Screenshots
13. Testing
14. Challenges Faced
15. Learnings
16. Future Enhancements
17. Conclusion
18. References

1. Introduction

The Smart Student Academic Management System is a web-based application developed to simplify the management of student information and academic records. The system provides a centralized platform for managing student details, subjects, marks, attendance and performance. It automatically calculates grades, average marks and GPA and provides performance analysis and student reports.

The application is implemented using Java and Spring Boot with Spring MVC and Thymeleaf. HTML and CSS are used for the web interface, while Java serialization provides persistent file-based storage for student and academic data. The project converts the functionality of a console-based academic management system into a browser-accessible application.

2. Problem Statement

Managing student academic information manually can be time-consuming and difficult to organize. Student details, marks, attendance and performance records may be maintained separately, making analysis and report generation inefficient. A centralized system is therefore required to store, validate and analyze academic information through a simple web interface.

The proposed system addresses this problem by combining student management, academic record management, performance analytics and report generation in one application.

3. Objectives

- To manage student information efficiently through a web interface.
- To maintain academic records including subjects, credits, marks and attendance.
- To calculate grades, average marks and GPA automatically.
- To identify weak subjects and attendance warnings.
- To generate a complete academic report for a selected student.
- To validate user input and prevent invalid academic data.
- To provide persistent storage so that saved data remains available after restarting the application.

4. Scope of the Project

- Student information management: add, search, update, display and delete.
- Academic record management: subjects, credits, marks and attendance.
- Performance analysis: grades, average marks, GPA and performance status.
- Weak-subject and attendance-warning identification.
- Student academic report generation.
- Persistent file-based storage using Java object serialization.
- Browser-based interaction through Spring Boot and Thymeleaf.

5. Functional Requirements

ID	Requirement	Description
FR1	Student Management	Add, search, update, display and delete student records.
FR2	Academic Management	Add subject, credits, marks and

		attendance records.
FR3	Grade Calculation	Calculate grade and grade point from marks.
FR4	Performance Analytics	Calculate average, GPA and performance status.
FR5	Warnings	Identify weak subjects and attendance below the defined threshold.
FR6	Reporting	Generate and view an academic report for a student.
FR7	Validation	Validate student data, marks, credits and attendance.
FR8	Persistence	Save and load student and academic records from files.

6. Non-Functional Requirements

Requirement	Implementation
Usability	Simple browser-based pages, clear forms and navigation.
Reliability	Validation and exception handling reduce invalid data and application errors.
Maintainability	Separate model, service, controller and exception packages.
Performance	Lightweight local file storage and direct calculations provide responsive operations.
Data Persistence	Student and academic records are stored in serialized files.
Error Handling	Invalid marks, attendance, credits and student details are rejected with messages.
Scalability	The modular structure allows database storage and authentication to be added later.
Security	Input validation and controlled server-side processing are used; future versions can add authentication and authorization.

7. System Modules

1. Student Management – Manages student registration number, name, email, program and semester. Supports add, search, update, display and delete operations.

2. Academic Management – Stores subject code, subject name, credits, marks and attendance for each student.

3. Performance Analytics – Calculates average marks, GPA, grades, performance status, weak subjects and attendance warnings.

4. Student Reports – Combines student details and academic records into a complete academic report.

5. Validation & Persistence – Validates input and stores data using Java serialization through dataManager.

8. Technology Stack

Technology	Purpose
Java 21	Core programming language and object-oriented implementation.
Spring Boot	Application framework and runtime.
Spring MVC	Controller-based web request handling.
Thymeleaf	Server-side HTML template rendering.
HTML5 / CSS3	Web page structure and styling.
Maven	Project build and dependency management through Maven Wrapper.
VS Code	Development environment.
Java Serialization	Persistent storage of students.dat and academic_records.dat.

9. System Architecture

The system follows a layered architecture. A user interacts with the application through a web browser. Spring Boot controllers receive HTTP requests and connect the web layer with the service and model classes. The dataManager class handles serialization and persistent file storage. The main persistent files are students.dat and academic_records.dat.

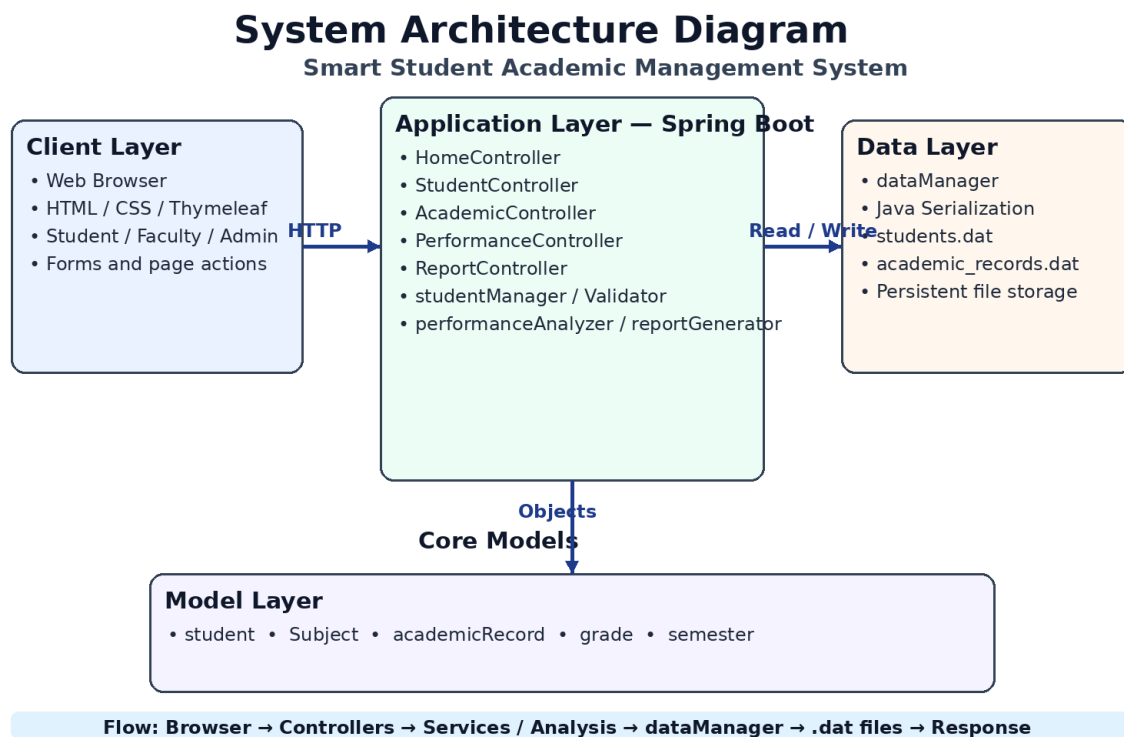


Figure 6. System Architecture of the Smart Student Academic Management System

Main request flow: Browser → Controller → Service / Analysis → Model and dataManager → Persistent files → Response.

10. UML and Design Views

10.1 Use Case View

Actor	Major Use Cases
Student / User	Open dashboard, manage student records, add academic records, view performance and view reports.
Faculty / Academic User	Maintain student academic data, inspect marks and attendance, analyze performance and reports.
System	Validate inputs, calculate grades/GPA, store data and generate reports.

10.2 Class / Component View

Class / Component	Responsibility
student	Stores registration number, name, email, program and semester.
Subject	Stores subject code, subject name and credits.
academicRecord	Associates a Subject with marks and attendance.
grade	Provides static grade and grade-point calculations.
studentManager	Handles student CRUD operations and persistence.
studentValidator	Validates student and academic input.
performanceAnalyzer	Calculates average, GPA, performance status, weak subjects and attendance warnings.
reportGenerator	Generates a formatted student academic report.
dataManager	Serializes and deserializes student and academic records.
Controllers	Connect browser requests to the application logic and templates.

10.3 Sequence / Workflow View

User → Browser: opens a module and submits an action

Browser → Controller: sends HTTP GET/POST request

Controller → Service/Manager: validates and processes the request

Service/Manager → Model/dataManager: creates or retrieves objects and records

dataManager → File System: reads/writes serialized data

Application → Controller → Browser: returns updated HTML page or report

10.4 Data Flow

Student information is stored in students.dat, while academic records are stored in academic_records.dat. Java Serializable objects are written using ObjectOutputStream and loaded using ObjectInputStream. The registration number acts as the key connecting a student with the associated academic records.

11. Implementation

The application was implemented by integrating the existing Java academic management logic into a Spring Boot web application. The generated Spring Boot project uses Maven and Java 21. The original model and service classes were organized under the `src.model`, `src.service` and `src.exception` packages, while web controllers were placed under `com.student.student_management`.

- HomeController loads the dashboard and current student count.
- StudentController handles student listing, add, edit and delete operations.
- AcademicController handles academic record creation and validation.
- PerformanceController prepares academic records for performance analysis.
- ReportController prepares complete student reports.
- studentManager performs student CRUD operations and persists student data.
- performanceAnalyzer calculates average, GPA, grades and warning information.
- dataManager provides serialized file storage.

11.1 Validation and Error Handling

Server-side validation checks that registration numbers, names, emails and programs are not empty, semester values are within range, credits are positive, marks are between 0 and 100, and attendance is between 0 and 100. Invalid values are returned to the form with an error message rather than being stored.

11.2 Data Persistence

The system uses Java object serialization instead of a relational database. Student objects are saved to `students.dat` and the academic record map is saved to `academic_records.dat`. This approach keeps the current project lightweight while demonstrating file handling, serialization, collections and object-oriented programming.

12. Application Screenshots

The following screenshots were captured from the running Spring Boot application at `localhost:8080`.

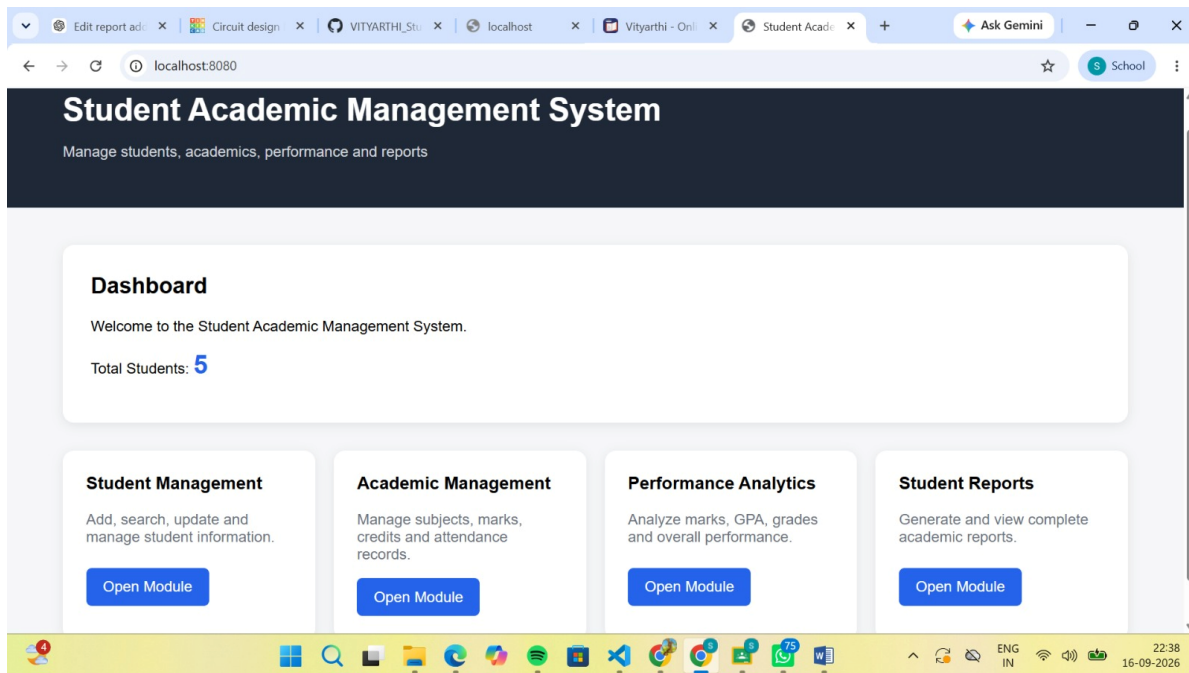


Figure 1. Application Dashboard

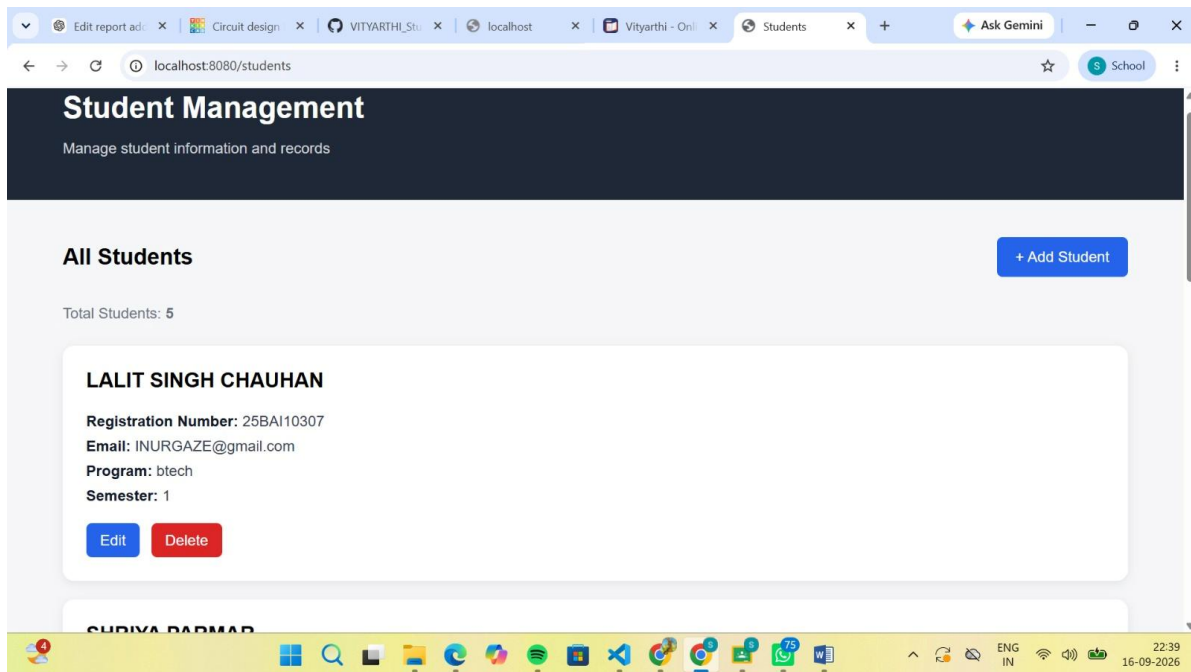


Figure 2. Student Management Module

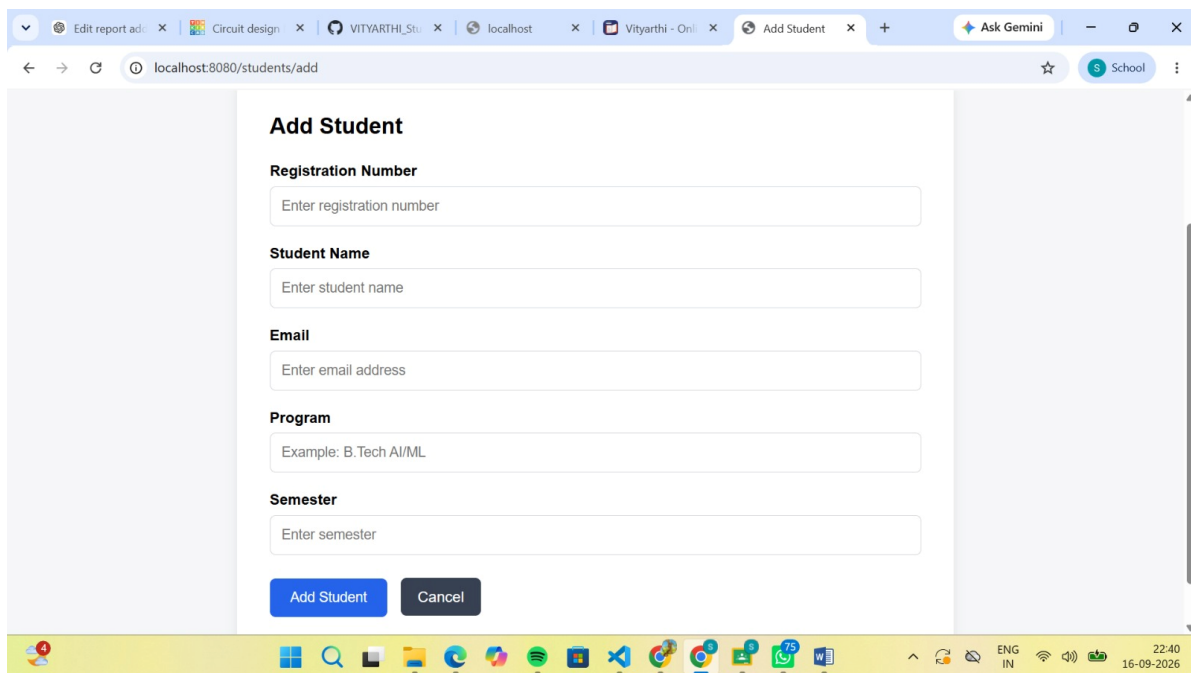


Figure 3. Add Student Form

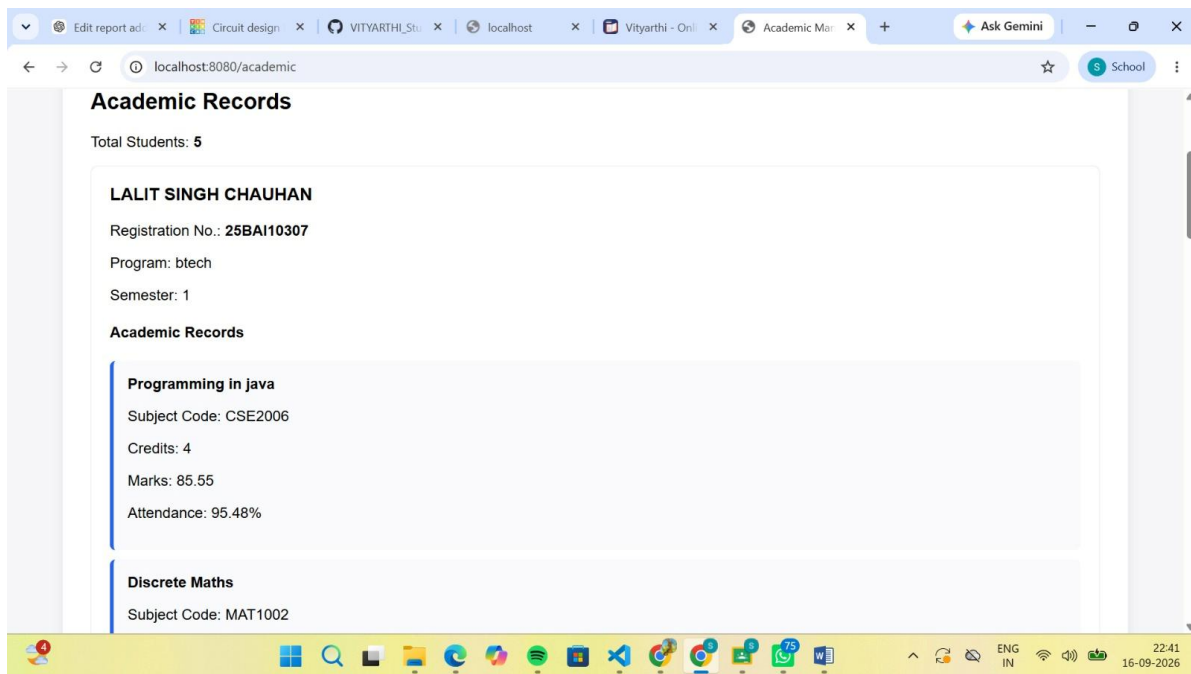


Figure 4. Academic Records Module

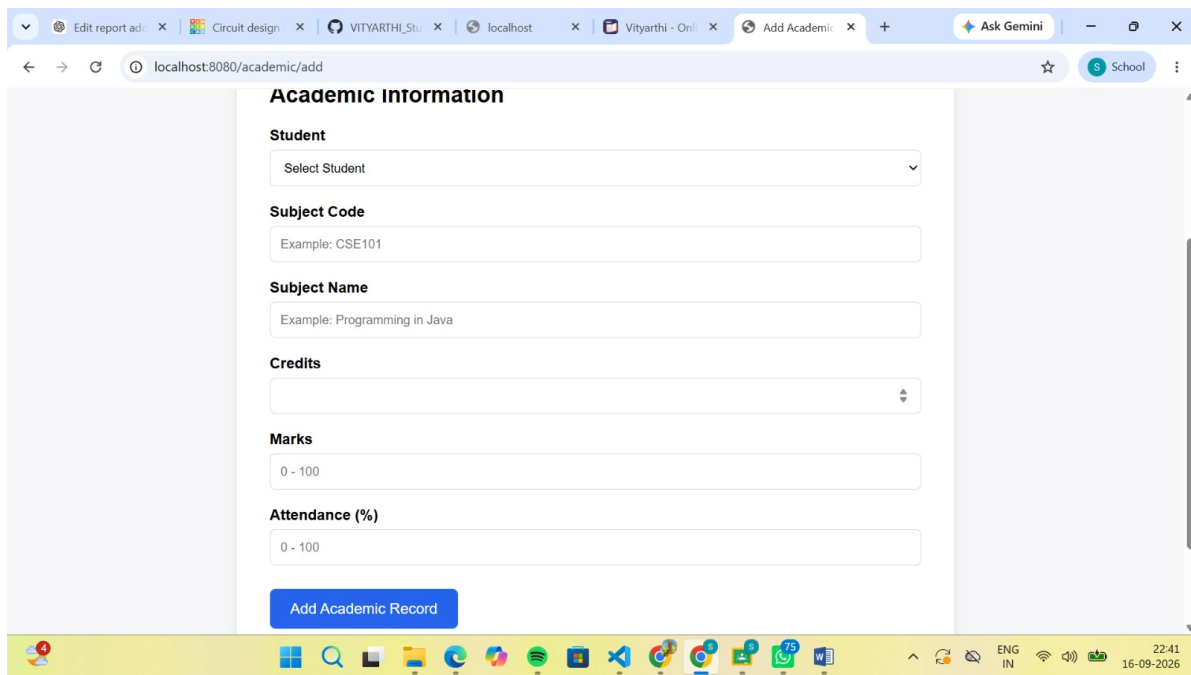


Figure 5. Add Academic Record Form

13. Testing

Test Case	Input / Action	Expected Result	Status
Add Student	Valid student details	Student added successfully	Passed
Edit Student	Updated student details	Student information updated	Passed

Delete Student	Registration number	Student removed	Passed
Search Student	Registration number	Matching student displayed	Passed
Add Academic Record	Valid marks and attendance	Record saved successfully	Passed
Invalid Marks	Marks > 100	Validation error displayed	Passed
Invalid Attendance	Attendance > 100	Validation error displayed	Passed
Performance Analysis	Student academic records	GPA, grades and status displayed	Passed
Student Report	Select student	Complete academic report generated	Passed
Data Persistence	Restart application	Saved records remain available	Passed

14. Challenges Faced

- Converting the existing console-based Java functionality into a Spring Boot web application.
- Managing package names and keeping the existing lowercase student class/file naming consistent.
- Configuring and running the project with Maven Wrapper because Maven was not available as a global command.
- Connecting Thymeleaf forms with Spring MVC controllers and RequestParam values.
- Ensuring that the latest student data is loaded correctly when pages are opened.
- Implementing validation and returning meaningful error messages for invalid academic inputs.
- Maintaining persistent student and academic records using serialized files.

15. Learnings

- Learned how Spring Boot applications are structured and executed using Maven.
- Understood the role of controllers in handling HTTP GET and POST requests.
- Learned how Thymeleaf connects Java model data with HTML pages.
- Applied object-oriented programming through model and service classes.
- Implemented collections such as ArrayList and HashMap for record management.
- Applied exception handling and input validation.
- Used Java serialization for persistent file storage.
- Understood how a console application can be transformed into a web-based application.

16. Future Enhancements

- Replace file-based storage with MySQL or another relational database.
- Add secure login and role-based access for students, faculty and administrators.
- Add search and filtering across academic records.
- Add semester-wise and subject-wise visual analytics.
- Allow report export to PDF.
- Add automated unit and integration testing.
- Deploy the application to a cloud platform for remote access.
- Add backup and restore functionality for academic data.

17. Conclusion

The Smart Student Academic Management System provides a centralized web-based solution for managing student and academic information. It combines CRUD operations, academic record management, validation, automated grade and GPA calculation, performance analysis and report generation in one application. The project demonstrates practical use of Java, object-oriented programming, Spring Boot, Spring MVC, Thymeleaf, file handling, serialization and exception handling. The modular design also provides a foundation for future improvements such as database integration, authentication, analytics dashboards and cloud deployment.

18. References

- VITYarthi – Build Your Own Project guidelines provided for the project.
- Spring Boot documentation and Spring MVC concepts.
- Java Platform documentation for collections, exception handling and object serialization.
- Thymeleaf documentation for server-side HTML templates.
- Apache Maven documentation and Maven Wrapper usage.

GitHub Repository: [VITYARTHI_Student-Academic-Management-System_JAVA](#)